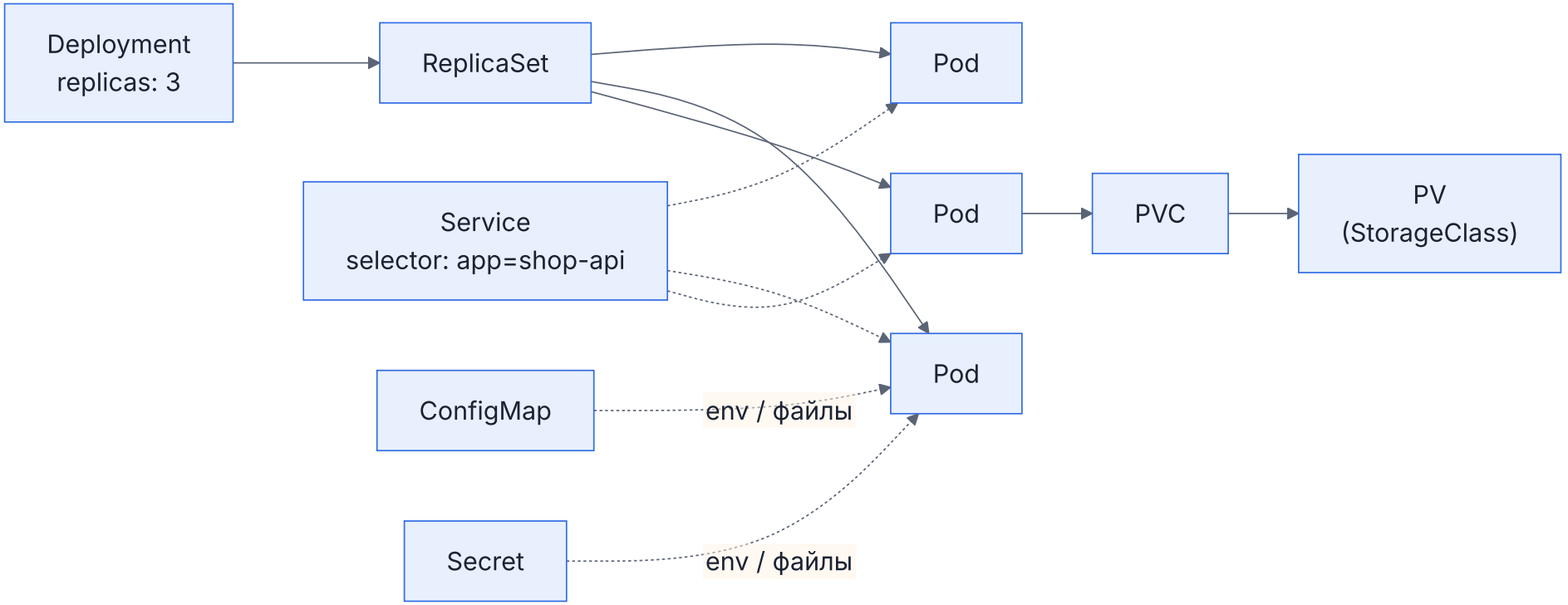
Kubernetes: основные сущности

Занятие 03 · День 2, утро

Pod, Deployment, Service, ConfigMap, Secret, тома и другие

Карта сущностей



Сегодня разберём каждый блок этой схемы и посмотрим вживую.

Namespace

- Логическое разделение кластера: команды, окружения, приложения
- Имена объектов уникальны **внутри** namespace
- На namespace вешаются **RBAC**, **ResourceQuota**, **LimitRange**, **NetworkPolicy**
- Не все объекты «в namespace»: Node, PV, StorageClass, ClusterRole — кластерные

```
kubectl get ns
kubectl create namespace demo
kubectl config set-context --current --namespace=demo   # namespace по умолчанию
kubectl api-resources --namespaced=false
```

Pod — минимальная единица

- Один или несколько контейнеров с **общей сетью** (один IP) и **общими томами**
- Под **смертен**: упал узел — под не «переезжает», вместо него создаётся новый (с новым IP)
- Руками поды почти не создают — ими управляют контроллеры

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web
  labels:
    app: web
spec:
  containers:
    - name: nginx
      image: nginx:1.28-alpine
      ports:
        - containerPort: 80
```

Жизненный цикл пода

Фаза / статус	Что значит
`Pending`	принят API, ещё не запущен: ждёт узла (scheduler) или скачивает образ
`ContainerCreating`	образ скачивается, монтируются тома
`Running`	хотя бы один контейнер работает
`Succeeded` / `Failed`	все контейнеры завершились (Job)
`CrashLoopBackOff`	контейнер падает, kubelet перезапускает с растущей паузой
`ImagePullBackOff`	не удалось скачать образ: имя, тег, доступ к registry

`restartPolicy`: `Always` (по умолчанию) · `OnFailure` · `Never`

Несколько контейнеров в поде

- **initContainers** — выполняются по очереди **до** основных: миграции БД, ожидание зависимостей
- **Sidecar** — вспомогательный контейнер рядом с основным: прокси, сборщик логов (с 1.33 стабильно: init-контейнер с `restartPolicy: Always`)

```yaml
spec:
  initContainers:
    - name: wait-db
      image: busybox:1.37
      command: ["sh", "-c", "until nc -z db 5432; do sleep 2; done"]
  containers:
    - name: app
      image: shop-api:1.0.0
```

> Правило: в один под — только то, что должно жить и масштабироваться **вместе**.

Labels, selectors, annotations

```yaml
metadata:
  labels:                       # для отбора: селекторы, kubectl -l
    app.kubernetes.io/name: shop-api
    app.kubernetes.io/part-of: shop
    tier: backend
  annotations:                  # для информации и инструментов, не для отбора
    description: "Каталог товаров"
    prometheus.io/scrape: "true"
```

```bash
kubectl get pods -l tier=backend
kubectl get pods -l 'app.kubernetes.io/name in (shop-api,shop-web)'
kubectl label pod web env=dev
kubectl get pods --show-labels
```

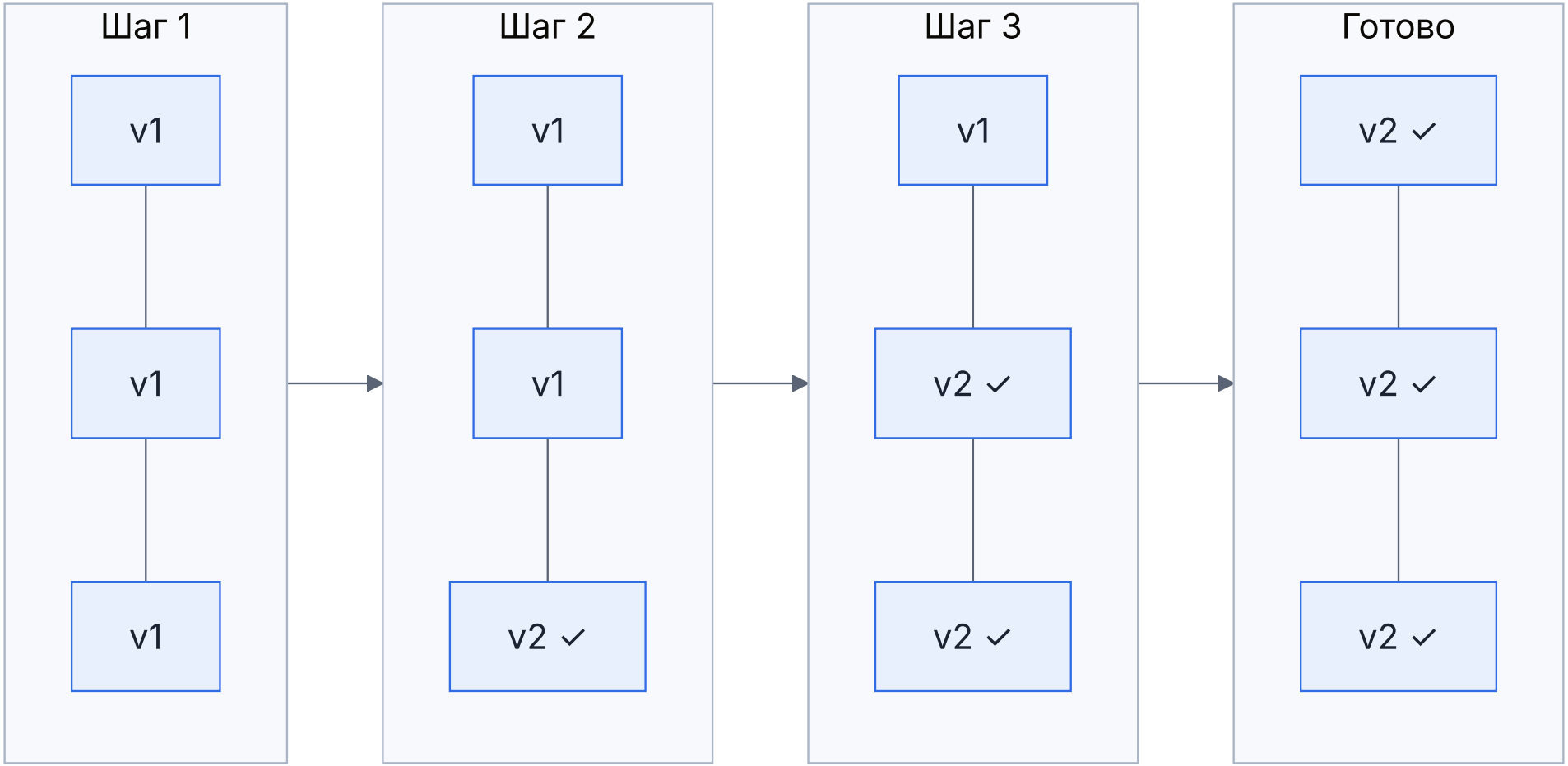
Метки — это «клей» Kubernetes: Service находит поды, ReplicaSet считает свои поды — всё по меткам.

ReplicaSet и Deployment

- **ReplicaSet** — держит N одинаковых подов по шаблону
- **Deployment** — управляет ReplicaSet'ами: обновления, откаты, история
- Работаем всегда с **Deployment**

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
spec:
  replicas: 3
  selector:
    matchLabels: { app: shop-api }
  template:                          # шаблон пода
    metadata:
      labels: { app: shop-api }      # должны совпадать с selector!
    spec:
      containers:
        - name: shop-api
          image: nginx:1.28-alpine
```

RollingUpdate: как это выглядит



Новые поды добавляются, старые удаляются — **по одному**, только когда новый прошёл readinessProbe.

Стратегии обновления Deployment

RollingUpdate (по умолчанию)

```yaml
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1        # +1 под сверху
    maxUnavailable: 0  # без потери мощности
```

Recreate — сначала удалить все, потом создать новые (простой!)

Нужна, если две версии не могут работать одновременно (миграции, RWO-том).

Другие контроллеры

Контроллер	Когда использовать	Пример
DaemonSet	по одному поду на каждом (или выбранном) узле	агенты: Calico, kube-proxy, сборщик логов
StatefulSet	стабильные имена, порядок, свой том у каждой реплики	PostgreSQL, Redis, Kafka, Gitaly
Job	задача до успешного завершения	миграция БД, бэкап
CronJob	Job по расписанию	ночной бэкап `0 3 * * *`

```
kubectl get ds -A                    # кто живёт на каждом узле
kubectl create job hello --image=busybox:1.37 -- echo "Привет"
kubectl create cronjob tick --image=busybox:1.37 --schedule="*/5 * * * *" -- date
```

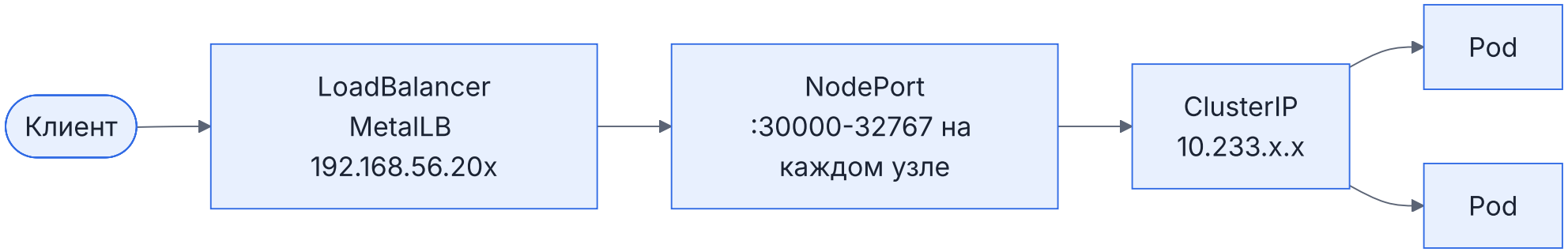
Service — стабильный адрес

- Поды приходят и уходят → у Service постоянные **IP** и **DNS-имя**
- Находит поды по **selector**, балансирует между **готовыми** (readiness!)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: shop-api
spec:
  type: ClusterIP
  selector:
    app: shop-api
  ports:
    - port: 80          # порт сервиса
      targetPort: 8080  # порт контейнера
```

DNS: `shop-api` (тот же ns) · `shop-api.shop` · `shop-api.shop.svc.cluster.local`

Типы Service



Тип	Доступен	У нас
`ClusterIP`	только внутри кластера	по умолчанию
`NodePort`	`<IP любого узла>:30000–32767`	работает
`LoadBalancer`	внешний IP от облака / MetalLB	`192.168.56.200–220`
`ExternalName`	CNAME на внешнее имя	—
headless (`clusterIP: None`)	DNS отдаёт IP подов	для StatefulSet

HTTP-вход: Ingress и Gateway API

Ingress (`networking.k8s.io/v1`)

- Маршрутизация по хосту/пути к Service
- Нужен контроллер
- Популярный **ingress-nginx прекращён** (retired) — новые проекты на нём не начинают

Gateway API (`gateway.networking.k8s.io/v1`)

- Преемник Ingress
- Роли: **GatewayClass** (инфраструктура) → **Gateway** (вход, порты) → **HTTPRoute** (маршруты приложения)
- Реализации: Envoy Gateway, Cilium, Istio, NGINX Gateway Fabric…

GitLab в нашем кластере (день 4) будет доступен через **Gateway API + Envoy Gateway**.

ConfigMap — конфигурация отдельно от образа

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: shop-config
data:
  GREETING: "Добро пожаловать!"
  LOG_LEVEL: info
  app.conf: |
    listen 8080
    cache on
```

```yaml
    containers:
      - name: app
        envFrom:
          - configMapRef: { name: shop-config }      # все ключи → переменные
        volumeMounts:
          - { name: conf, mountPath: /etc/app }      # ключи → файлы
    volumes:
      - name: conf
        configMap: { name: shop-config }
```

Secret

- То же, что ConfigMap, но для **паролей, токенов, ключей, сертификатов**
- В манифесте — **base64**. Это **кодирование, не шифрование!**
- Защита: RBAC на чтение Secret, шифрование etcd at rest, внешние хранилища (Vault, External Secrets, Sealed Secrets)

```
kubectl create secret generic db-cred \
  --from-literal=username=shop --from-literal=password='S3cr3t!'
kubectl get secret db-cred -o yaml
echo 'UzNjcjN0IQ==' | base64 -d
```

Типы: `Opaque`, `kubernetes.io/dockerconfigjson` (доступ к registry), `kubernetes.io/tls`, `kubernetes.io/service-account-token`

Хранилище: Volume, PV, PVC, StorageClass

Объект	Кто создаёт	Что это
`volumes:` в поде	разработчик	`emptyDir`, `configMap`, `secret`, `persistentVolumeClaim`
PersistentVolumeClaim	разработчик	«Мне нужно 5Gi, RWO»
StorageClass	администратор	«как создавать диски»: local-path, Ceph, облачные диски
PersistentVolume	provisioner автоматически	конкретный диск

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data }
spec:
  accessModes: [ReadWriteOnce]
  resources: { requests: { storage: 1Gi } }
  # storageClassName не указан → default (у нас local-path)
```

Ресурсы: requests и limits

```
resources:
  requests: { cpu: 100m, memory: 128Mi }   # гарантия; по ним планирует scheduler
  limits:   { cpu: 500m, memory: 256Mi }   # потолок
```

- **CPU** сверх лимита → троттлинг (медленно, но живёт)
- **Память** сверх лимита → **OOMKilled** (контейнер убит)
- `100m` = 0.1 ядра; `Mi` = 2^{20} байт

QoS-класс	Условие	Выселяется при нехватке памяти
Guaranteed	requests = limits	последним
Burstable	requests < limits	средне
BestEffort	ничего не задано	первым

Пробы (probes)

Проба	Вопрос	Неудача →
`startupProbe`	приложение уже запустилось?	ждём, остальные пробы не работают
`readinessProbe`	готово принимать трафик?	под убирается из Service
`livenessProbe`	живо ли?	контейнер **перезапускается**

```yaml
readinessProbe:
  httpGet: { path: /healthz, port: 8080 }
  periodSeconds: 5
livenessProbe:
  httpGet: { path: /healthz, port: 8080 }
  initialDelaySeconds: 10
```

> Liveness не должна зависеть от БД: упала база — перезапуск всех подов не поможет.

ServiceAccount и RBAC — кратко

- **ServiceAccount** — «учётка» для программ в кластере (у подов, у CI)
- **Role / ClusterRole** — что можно: `verbs` над `resources`
- **RoleBinding / ClusterRoleBinding** — кому выдать

```yaml
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata: { name: pod-reader, namespace: demo }
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
```

```
kubectl auth can-i list pods -n demo --as=system:serviceaccount:demo:default
```

Пригодится на 5-й день: доступ GitLab CI к кластеру.

ДЕМО kubectl: команды на каждый день

Действие	Команда
Список	`kubectl get pods -o wide`, `get all -n demo`, `get pods -w`
Подробно + события	`kubectl describe pod web`
События namespace	`kubectl get events --sort-by=.lastTimestamp`
Логи	`kubectl logs web`, `-f`, `--previous`, `-c <контейнер>`, `-l app=web`
Внутрь контейнера	`kubectl exec -it web -- sh`
Проброс порта	`kubectl port-forward svc/shop-api 8080:80`
Отладка	`kubectl debug -it web --image=busybox:1.37 --target=nginx`
Документация	`kubectl explain pod.spec.containers.resources`
YAML объекта	`kubectl get deploy web -o yaml`

ДЕМО Демо: собираем всё вместе в namespace `demo`

```
kubectl create namespace demo
kubectl -n demo create deployment web --image=nginx:1.28-alpine --replicas=3
kubectl -n demo get deploy,rs,pods -o wide --show-labels

kubectl -n demo delete pod <любой-под>     # контроллер создаст новый
kubectl -n demo scale deploy web --replicas=5

kubectl -n demo expose deploy web --port=80
kubectl -n demo run tmp --rm -it --image=busybox:1.37 -- wget -qO- web

kubectl -n demo create configmap web-cfg --from-literal=MODE=demo
kubectl -n demo set env deploy/web --from=configmap/web-cfg
kubectl -n demo rollout history deploy/web
```

Итоги

- **Pod** — единица запуска; управляют ими **Deployment / StatefulSet / DaemonSet / Job**
- **Метки и селекторы** связывают всё
- **Service** даёт стабильный адрес, **Gateway API / Ingress** — HTTP-вход
- **ConfigMap / Secret** — конфигурация отдельно от образа
- **PVC + StorageClass** — данные, переживающие под
- **requests/limits** и **пробы** — обязательны для production

Вопросы

1. Чем readiness отличается от liveness?
2. Почему Secret в base64 — это не защита?
3. Какой контроллер выбрать для PostgreSQL? Для агента логов?

Практика 03

Сущности Kubernetes на практике

Задание — на платформе testlms.kz